РЕФЕРАТ

Расчетно-пояснительная записка 25 с., 5 рис., 0 табл., N источн., N прил.

БАЗЫ ДАННЫХ, POSTGRESQL, СИСТЕМЫ УПРАВЛЕНИЯ ПРОЕКТАМИ, СУП, БД, REDMINE

Объектом исследования являются база данных для системы управления проектами

Цель работы — разработать базу данных и программное обеспечение, обслуживающее эту базу для данных, для системы управления базы данных.

В ходе работы TODO написать

СОДЕРЖАНИЕ

РЕФЕРАТ 4

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ 7

ВВЕДЕНИЕ 8

1 Аналитический раздел 10

1.1 Описание сущностей средства управления проектами redmine 10

1.1.1 Сущность `projects` 11

1.1.2 Сущность `users` 11

1.1.3 Сущность `issues` 12

1.1.4 Сущность `trackers` 12

1.1.5 Сущность `issue_statuses` 12

1.1.6 Сущность `roles` 13

1.1.7 Сущность `members` 13

1.1.8 Сущность `custom_fields` 13

1.1.9 Сущность `journals` 14

1.1.10 Сущность `attachments` 14

1.1.11 Сущность `wikis` 14

1.1.12 Сущности коммуникаций 15

1.1.13 Сущности учета времени 15

1.1.14 Остальные сущности 15

1.2 Описание типовых процессов и ролей в redmine, связанные с учебным процессом 16

1.2.1 Процесс выдачи задачи 17

1.2.2 Процесс выполнения задачи в учебном процессе . . . 18

1.2.3 Прооцесс проверки задачи 18

1.3 Анализ существующих решений в области СУБД 19

1.3.1 Иерархическая модель данных 19

1.3.2 Сетевая модель данных 20

1.3.3 Реляционная модель данных 20

1.3.4 Документоориентированная модель данных 22

1.3.5 Объектно-ориентированная модель данных 22
1.3.6 Колоночная модель данных 23
1.3.7 Графовая модель данных 23
1.3.8 Преимущества и недостатки 23

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 25

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

СУП — система управления проектами.

БД — база данных.

ВВЕДЕНИЕ

База данных является неотъемлемой частью любого программного обеспечения, где требуется хранить данные пользователя. От базы данных напрямую зависит время выполнения запросов, использующих сохраненные данные, что критично для систем с высокой нагрузкой. В системах управления проектами объем информации велик: необходимо ежедневно отслеживать статусы задач, сроки исполнения, приоритеты и взаимодействие между членами команды. Неоптимальная структура таблиц приводит к избыточности данных, нарушению ссылочной целостности и замедлению формирования аналитических отчетов. Поэтому грамотное проектирование схемы хранения становится ключевым фактором производительности всего приложения и обеспечения надежности бизнес-процессов.

Целью работы является разработка реляционной базы данных для системы управления проектами.

Для достижения поставленной цели необходимо решить следующие задачи:

В аналитической части:

1) Описать типовые процессы и роли в redmine, связанные с учебным процессом.
2) Описать сущности средства управления проектами redmine.
3) Проанализировать существующие решения в области СУБД.

В конструкторской части:

1) Спроектировать базу данных.
2) Проверить, что разработанная база данных находится в 3НФ.
3) Разработать хранимые процедуру.
4) Разработать интеграции с внешним программным обеспечением для расширения функциональности.

В технологической части:

1) Обосновать выбор целевой платформы и средств разработки.
2) Привести примеры реализации хранимых процедур, разработанных скриптов.
3) Привести инструкции по развертыванию программы.

В исследовательской части:

1) Исследовать зависимость времени выполнения операций в зависимости от количества входящих запросов.
2) Исследовать зависимость времени выполнения операций в зависимости от кэширования.

1 Аналитический раздел

1.1 Описание сущностей средства управления проектами redmine

Система управления проектами Redmine представляет собой реляционную базу данных, состоящую из более чем 50 взаимосвязанных таблиц. Модель базы данных сервиса redmine [1] представлена на рисунке 1.

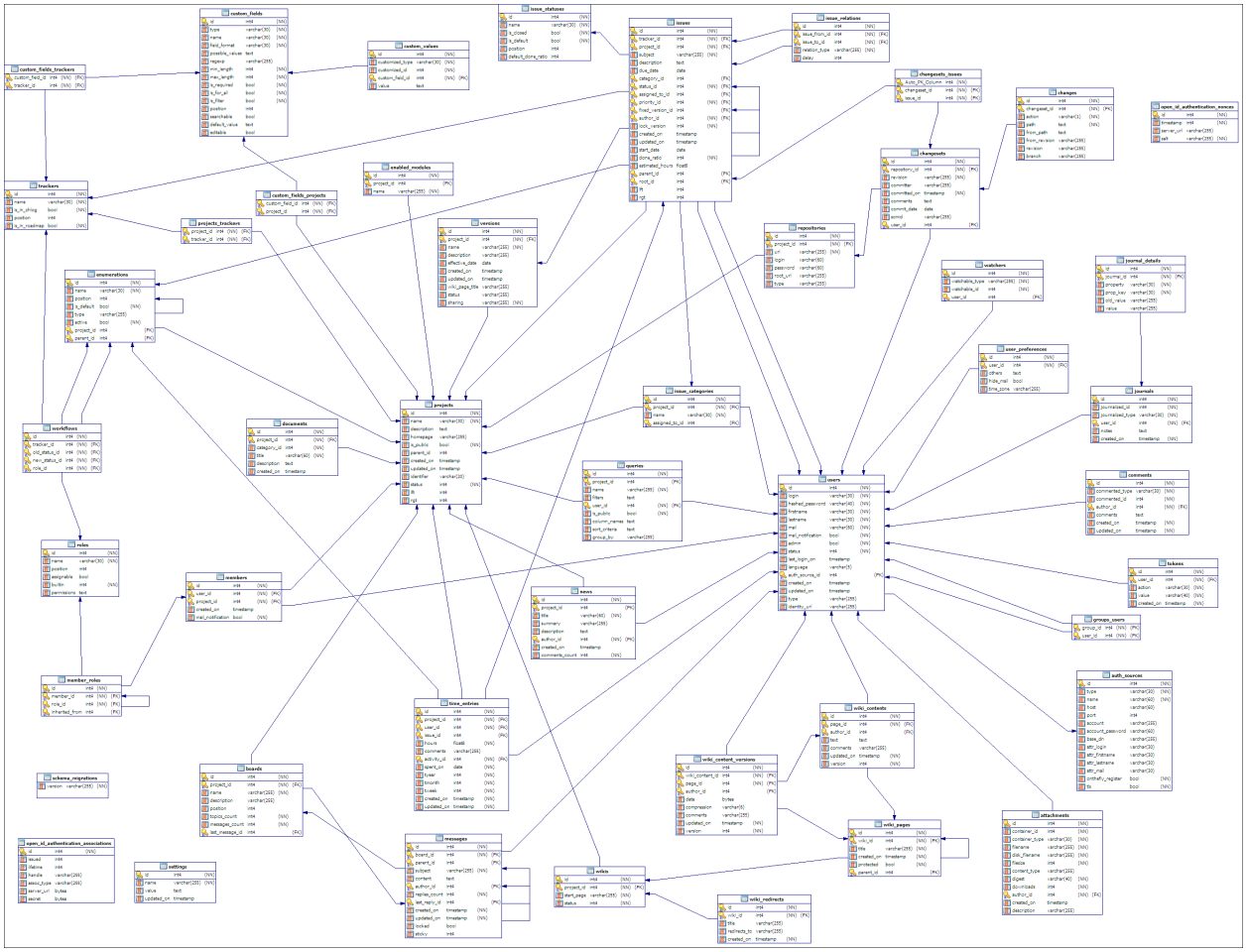

Рисунок 1 – Диаграмма базы данных сервиса redmine

В диаграмме выделяются следующие сущности, которые можно назвать основными:

— ядро системы (проекты, пользователи, задачи, настраиваемые поля);
— управление доступом и ролями;
— отслеживание изменений и версионирование;

— документооборот, вики и коммуникация (форумы, новости, комментарии);
— учет времени;

В последующих подглавах, будет описаны основные сущности.

1.1.1 Сущность `projects`

В системе управления проектами redmine проект является основной сущностью она содержит в себе следующую информацию:

— `id` — уникальный идентификатор проекта;
— `name` — наименование проекта;
— `description` — текстовое описание проекта;
— `homepage` — URL домашней страницы проекта;
— `is_public` — флаг публичности проекта;
— `parent_id` — идентификатор родительского проекта;
— `created_on` и `updated_on` — временные метки;
— `status` — статус проекта (активен/архивирован);

Благодаря полю `parent_id`, появляется возможность создавать иерархию проектов.

1.1.2 Сущность `users`

Таблица `users` хранит информацию о пользователях системы, и содержит в себе следующие поля:

— `id` — уникальный идентификатор пользователя;
— `login` — логин пользователя;
— `hashed_password` — хешированный пароль;
— `firstname`, `lastname` — имя и фамилия;
— `mail` — адрес электронной почты;
— `admin` — флаг администратора системы;
— `status` — статус пользователя (активен/заблокирован);
— `last_login_on` — время последнего входа;
— `created_on`, `updated_on` — временные метки создания и обновления.

1.1.3 Сущность `issues`

Таблица `issues` представляет основную рабочую единицу системы — задачу:

— `id` — уникальный идентификатор задачи;
— `tracker_id` — тип задачи;
— `project_id` — проект, к которому относится задача;
— `subject` — название задачи;
— `description` — описание задачи;
— `due_date` — дедлайн;
— `category_id` — категория задачи;
— `status_id` — статус задачи;
— `assigned_to_id` — исполнитель задачи;
— `priority_id` — приоритет задачи;
— `fixed_version_id` — версия, в которой будет выполнена;
— `author_id` — автор задачи;
— `created_on`, `updated_on` — временные метки;
— `start_date` — дата начала выполнения;
— `done_ratio` — процент выполнения;
— `estimated_hours` — оценочное время в часах;
— `parent_id` — родительская задача;
— `root_id` — корневая задача в дереве.

1.1.4 Сущность `trackers`

Таблица `trackers` определяет типы задач:

— `id` — идентификатор;
— `name` — имя типа;

1.1.5 Сущность `issue_statuses`

Таблица `issue_statuses` содержит возможные статусы задач:

— `id` — идентификатор;
— `name` — наименование статуса;
— `is_closed` — флаг закрытого статуса;

— `is_default` — флаг статуса по умолчанию;
— `default_done_ratio` — процент выполнения по умолчанию.

1.1.6 Сущность `roles`

Таблица `roles` определяет роли пользователей в системе:

— `id` — идентификатор;
— `name` — название роли;
— `assignable` — возможность назначения роли;
— `permissions` — текстовое поле с правами доступа.

1.1.7 Сущность `members`

Таблица `members` связывает пользователей с проектами:

— `id` — уникальный идентификатор;
— `user_id` — идентификатор пользователя;
— `project_id` — идентификатор проекта;
— `created_on` — дата добавления;
— `mail_notification` — настройки уведомлений.

Таблица `member_roles` связывает участников проекта и их роли.

1.1.8 Сущность `custom_fields`

Таблица `custom_fields` определяет пользовательские поля:

— `id` — идентификатор сущности;
— `type` — тип поля;
— `name` — название поля;
— `field_format` — формат поля;
— `possible_values` — возможные значения;
— `is_required` — обязательность заполнения;
— `searchable` — возможность поиска;
— `default_value` — значение по умолчанию;
— `editable` — возможность редактирования.

Таблица `custom_values` хранит значения настраиваемых полей.

1.1.9 Сущность `journals`

Таблица `journals` хранит историю изменений задач:

— `id` — уникальный идентификатор;

— `journable_id` — идентификатор изменяемого объекта;

— `journable_type` — тип объекта;

— `user_id` — пользователь, внесший изменения;

— `notes` — текстовые заметки;

— `created_on` — дата изменения.

Основная таблица `journals` также связана со вспомогательными:

— `journal_details` содержит детали каждого изменения;

— `changes` хранит информацию об измененных файлах;

— `changesets_issues` связывает коммиты с задачами.

1.1.10 Сущность `attachments`

Таблица `attachments` хранит информацию о вложенных файлах:

— `id` — уникальный идентификатор;

— `container_id` — идентификатор объекта-контейнера;

— `container_type` — тип контейнера;

— `filename` — имя файла;

— `disk_filename` — имя файла на диске;

— `filesize` — размер файла;

— `content_type` — MIME-тип;

— `digest` — хеш файла;

— `downloads` — количество скачиваний;

— `author_id` — автор загрузки;

— `created_on` — дата загрузки;

— `description` — описание файла.

Таблица `documents` хранит метаданные документов.

1.1.11 Сущность `wikis`

Таблица `wikis` определяет вики для проектов:

— `id` — уникальный идентификатор;

— `project_id` — проект вики;
— `start_page` — стартовая страница;
— `status` — статус вики.

Таблица `wiki_pages` содержит информацию о страницах. Таблица `wiki_contents` хранит текст страниц.

1.1.12 Сущности коммуникаций

К сущностям коммуникаций принадлежат следующие таблицы:

1) `news` хранит новостные сообщения проектов;
2) `boards` определяет форумы проектов;
3) `messages` хранит сообщения форумов;
4) `comments` хранит комментарии к различным объектам;

1.1.13 Сущности учета времени

Для учета времени, затраченного на задачу, используется таблица `time_entries` фиксирует затраченное время. Она обладает следующими полями:

— `id` — уникальный идентификатор;
— `project_id` — проект;
— `user_id` — пользователь;
— `issue_id` — задача;
— `hours` — количество часов;
— `comments` — комментарий;
— `activity_id` — тип активности;
— `spent_on` — дата работы;
— `tweek` — неделя;
— `tmonth` — месяц;
— `tyear` — год;
— `created_on`, `updated_on` — временные метки.

1.1.14 Остальные сущности

К остальным таблицам относятся:

— `watchers` позволяет пользователям подписываться на изменения задач;
— `repositories` хранит информацию о репозиториях;
— `enumerations` хранит справочные значения;
— `issue_categories` определяет категории задач;
— `queries` хранит сохраненные пользовательские фильтры;
— `workflows` определяет правила перехода статусов;
— `user_preferences` хранит предпочтения пользователей;
— `settings` содержит глобальные настройки;
— `enabled_modules` определяет активные модули проектов;
— `tokens` хранит токены для различных операций;
— `groups_users` связывает группы с пользователями;
— `versions` хранит информацию о версиях продукта.

1.2 Описание типовых процессов и ролей в redmine, связанные с учебным процессом

В текущем учебном процессе можно выделить следующие сущности:
— задача;
— система управления проектами;
— студент;
— преподаватель;
— система управления проектами (gitlab).

Объекты, которые взаимодействует с данными сущностями:
— генератор задач;
— администратор системы управления проектами;
— скрипт для создания задач в системе управления проктами;
— TODO gitlab bot.

Были выделены следубщие процессы:
1) генерация задачи и запись ее в базу данных задач.
2) **Администратор СУП:** запуск сервиса на сервере, создание пользователей, настройка трекеров и статусов, создание проекта, добавление пользователей в проект.
3) **Создание задачи:** скрипт достает задачу из базы данных и добавляет ее в систему управления проектами.

4) **Выполнение задачи:**

5) **Проверка задачи:**

На основании текущего учебного процесса была построена BPMN 2.0 диаграмма, представленная на рисунке 2

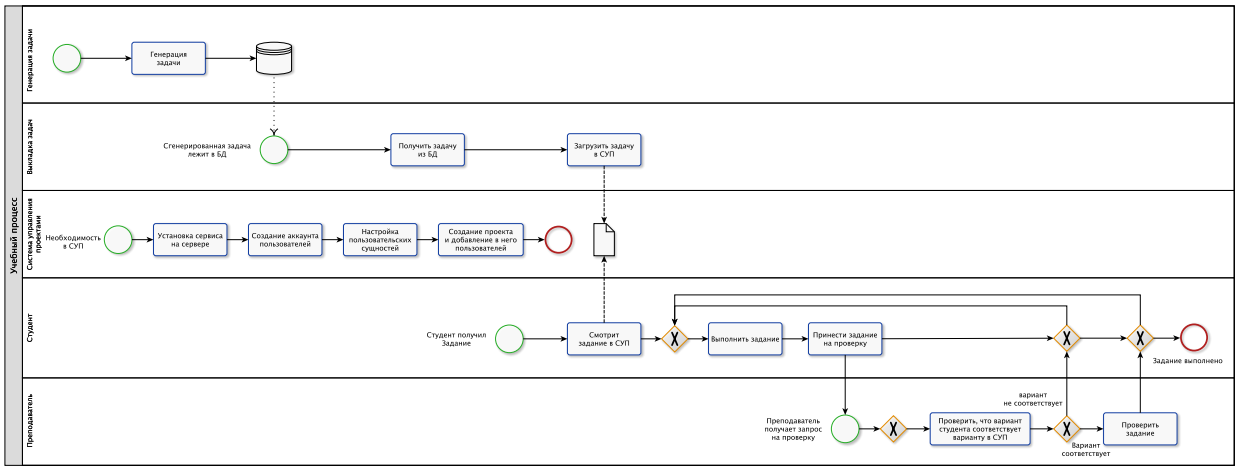

Рисунок 2 – Диаграмма BPMN-2.0 учебного процесса

1.2.1 Процесс выдачи задачи

Для выдачи задания, админом или преподавателем создается новый проект, в описании которого пишется задание, а после создаются `issue` для каждого студента, где в описание записываются вариант задания, а в поле `assignee` указывается студент. Процесс выдачи задания демонстрируется на рисунке 3.

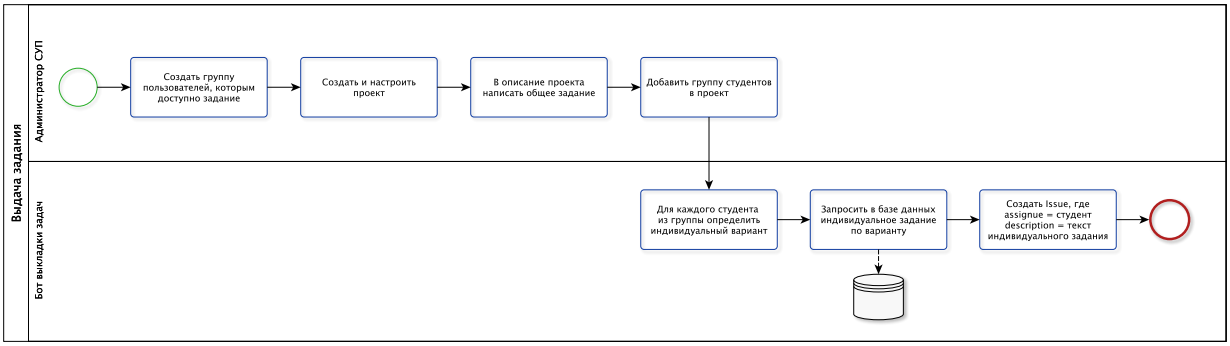

Рисунок 3 – Диаграмма BPMN-2.0 процесса выдачи задания

1.2.2 Процесс выполнения задачи в учебном процессе

Процесс выполнения задачи происходит с устного уведомления о выдаче задания. Далее студент открывает систему управления версиями, заходит в проект, соответствующий названию лабараторной работы. После получения задания, студент создает ветку на `gitlab`, куда загружает решение задачи. После создается `Merge Request`, в название которого включается номер задачи из системы управления версиями. По номеру задания `gitlab bot` связывает задачу и статус `merge request‘a` Процесс выполнения задания демонстрируется на рисунке 4.

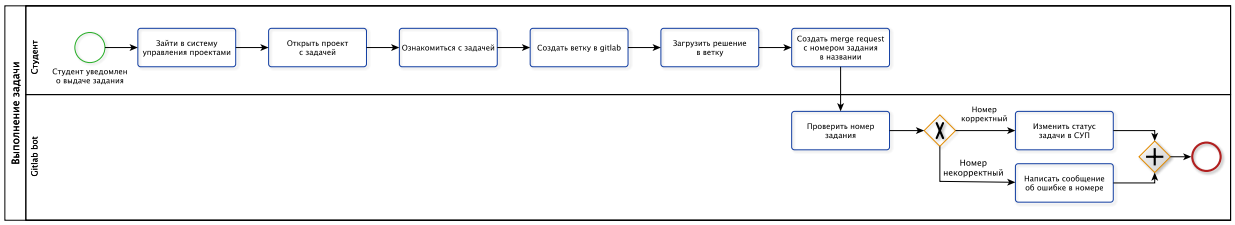

Рисунок 4 – Диаграмма BPMN 2.0 процесса выполнения задания

1.2.3 Прооцесс проверки задачи

Для проверки задачи студент приносит выполненную лабораторную работу на пару, и показывает преподавателю `merge request`. Преподаватель открывает задание в redmine и сравнивает выданный вариант с фактическим. После этого начинается процесс проверки задания, если задание выполнено успешно, то `merge request`, если нет, то будут выданы правки в комментарии. Если `Merge Request` закрыт, то `gitlab bot` переводит задачу в статус `done`. Процесс проверки задания демонстрируется на рисунке 5.

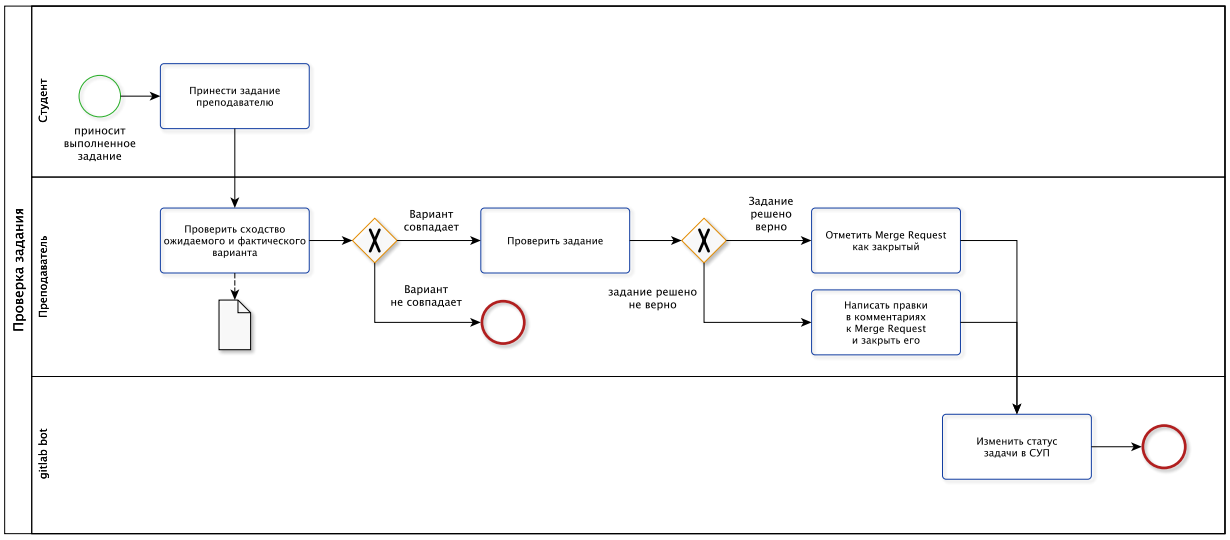

Рисунок 5 – Диаграмма BPMN 2.0 процесса выполнения задания

1.3 Анализ существующих решений в области СУБД

Система управления базами данных (СУБД) — приложение, обеспечивающее создание, хранение, обновление и поиск информации в базах данных. СУБД классифицируются следующими по нескольким признакам:

— **по модели данных**: иерархическая, сетевая, документоориентированная, реляционная, объектно-ориентированная, колоночная, графовая;

— **по архитектуре организации хранения данных**: локальные, распределённые;

— **по способу доступа к данным**: файл-серверные, встраиваемые, клиент-серверные, сервисно-ориентированные.

В данном разделе будет приведена информация о моделях данных в базах данных.

1.3.1 Иерархическая модель данных

Иерархическая модель данных стала первой широко распространенной моделью организации данных в коммерческих СУБД. Данные представляются в виде дерева. Связи между данными реализуются через физические указатели и представляют собой отношения типа «один-ко-многим».

Иерархическая модель используется для отображения иерархической

структуры работ. Доступ к данным осуществляется путем навигации от корня дерева вниз по уровням вложенности. При работе с иерархическими базами данных выделяют следующие приемущества и недостатки:

преимущества:

1) поиск в дереве имеет алгоритмическую сложность $O(logn)$.

Недостатки:

1) невозможна реализации связи «многие-ко-многим» без дублирования данных.

1.3.2 Сетевая модель данных

Сетевая модель данных [2] была разработана как расширение иерархической модели для устранения ее основных недостатков. В сетевой модели запись может иметь несколько родителей, что позволяет организовывать данные в виде произвольного графа. Данные организовуются в виде наборов, состоящих из владельца и членов. У сетевой модели данных можно выделить следующие преимущества и недостатки:

преимущества:

1) возможность представления отношения «многие-ко-многим».

Недостатки:

1) устаревание технологии.

1.3.3 Реляционная модель данных

В настоящее время реляционная модель данных, предложенная Э.Ф. Коддом в 1970 году, является доминирующей на рынке СУБД [3]. Данные представляются в виде набора отношений (таблиц), каждое из которых состоит из строк (кортежей) и столбцов (атрибутов). В реляционных СУБД связи между таблицами устанавливаются логически с помощью первичных и вторичных ключей.

— **Первичный ключ (Primary Key):** уникально идентифицирует запись в таблице.

— **Внешний ключ (Foreign Key):** ссылается на первичный ключ другой таблицы, обеспечивая ссылочную целостность.

Для взаимодействия используется структурированный язык запросов

SQL (Structured Query Language), который состоит из следующих языков:

- **DDL** (Data Definition Language) определяет структуру базы данных, позволяя создавать, изменять и удалять объекты.
- **DML** (Data Manipulation Language) управляет содержимым таблиц, обеспечивая вставку, обновление и удаление записей.
- **DQL** (Data Query Language) предназначен исключительно для выборки и чтения данных из базы без их изменения.
- **DCL** (Data Control Language) регулирует права доступа пользователей к объектам базы данных с помощью команд предоставления и отзыва привилегий.
- **TCL** (Transaction Control Language) контролирует целостность транзакций, позволяя подтверждать или откатывать группы операций.

Реляционный базы данных обладают фундаментальными свойствами ACID, которые помогают избежать неопределенного поведения при работе с данными:

- **Atomicity** (Атомарность): Транзакция выполняется целиком или не выполняется вовсе; не допускается частичное выполнение.
- **Consistency** (Согласованность): Транзакция переводит базу данных из одного согласованного состояния в другое, сохраняя целостность данных.
- **Isolation** (Изолированность): Параллельные транзакции не влияют друг на друга, обеспечивая независимость выполнения.
- **Durability** (Долговечность): Успешно завершенная транзакция сохраняется даже при сбоях оборудования.

У реляционных баз данных выделяют следующие преимущества и недостатки: **преимущества:**

1) **целостность данных (ACID):** свойства ACID обеспечивают базе данных целостность данных, помогая избегать конфликтов;
2) **нормализация:** возможность приведения базы данных к нормальным формам для устранения избыточности и аномалий обновления.

Недостатки:

1) изменение структуры таблиц на больших объемах данных может

блокировать работу системы;

2) при чрезмерной нормализации и большом количестве связей запросы с множественными соединениями выполняются медленнее.

В данный момент наибольшей популярностью пользуются следующие реляционные базы данных: PostgreSQL, MySQL, Oracle, MS SQL Server.

1.3.4 Документоориентированная модель данных

Документоориентированные СУБД относятся к классу NoSQL. Данные хранятся в виде документов, где каждый документ содержит данные в виде пар «ключ-значение» или вложенных структур. В таких системах (MongoDB, Couchbase, RavenDB) отсутствует жесткая схема данных. Документы в одной коллекции могут иметь различный набор полей. Данные часто денормализованы: связанная информация может храниться внутри одного документа. Выделяют следующие преимущества и недостатки:

преимущества:

1) можно добавлять новые атрибуты к задачам без миграции всей базы данных;
2) высокая скорость вставки данных благодаря отсутствию проверок внешних ключей и транзакционных блокировок на уровне строк.

Недостатки:

1) **сложность связей:** реализация связей между документами менее эффективна, чем в реляционной моделе данных;
2) дублирование данных.

1.3.5 Объектно-ориентированная модель данных

Объектно-ориентированные СУБД (OODBMS) предназначены для хранения данных в формате, совместимом с объектно-ориентированными языками программирования. Данные хранятся в виде объектов. Эта модель устраняет необходимость в объектно-реляционном отображении. Выделяют следующие преимущества и недостатки:

преимущества:

1) нет необходимости преобразовывать объекты языка программирования в строки таблиц и обратно;

2) возможность хранения наследования и сложных структур данных непосредственно в базе.

Недостатки:

1) отсутствие универсального языка запросов, сравнимого с SQL;
2) привязка к языку;

К объектным базам данных относятся: db4o, ObjectDB, Versant.

1.3.6 Колоночная модель данных

Колоночные СУБД хранят данные не построчно, а по столбцам. Все значения одного атрибута хранятся вместе на диске. Такая организация позволяет эффективно сжимать данные и быстро выполнять агрегацию, что полезно в целях аналитики. К колоночным СУБД относятся: ClickHouse, Vertica, Amazon Redshift. Выделяют следующие преимущества и недостатки:

преимущества:

1) быстрая скорость выполнения операций выборки, так как данные из столбцы лежат последовательно;
2) колоночное хранения обеспечивают возможность сжатия данные, что позволяет экономить дисковое пространство.

Недостатки:

1) получение полной информации о сущености (всех полей) требует сборки данных из разных мест диска, что замедляет выполнение операции.

1.3.7 Графовая модель данных

Графовые СУБД специализируются на хранении и обработке связей между данными. Данные представляются в виде узлов, ребер (связи) и свойств. Связи являются объектами первого класса и хранятся явно, а не вычисляются при запросе. У графовой модели данных выделяют следующие преимущества и недостатки:

1.3.8 Преимущества и недостатки

преимущества:

1) поиск путей, зависимостей и связей выполняется за константное время;
2) модель данных визуально соответствует реальным зависимостям.

Недостатки:

1) сложность масштабирования, так как распределение графа между серверами является сложной алгоритмической задачей;
2) неэффективны для простых запросов, не задействующих связи.

Графовые базы данных используются в проектах, где объекты связаны между собой, например в социальных сетях. Наиболее популярные графовые базы данных: Neo4j, ArangoDB, Amazon Neptune.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. modelo-1.0.1.png. https://www.redmine.org/attachments/4875/modelo-1.0.1.png. Дата обращения: 09.03.2026.

2. Е.В. Колпакова. СЕТЕВАЯ МОДЕЛЬ БАЗ ДАННЫХ // Студенческий научный форум. 2017.

3. ЦСР. Рынок систем управления и обработки данных в Российской Федерации:. https://www.csr.ru/upload/iblock/26a/swogkcus54ne2jaqcn5r98cq8hiu4d3p.pdf. Дата обращения: 04.03.2026.